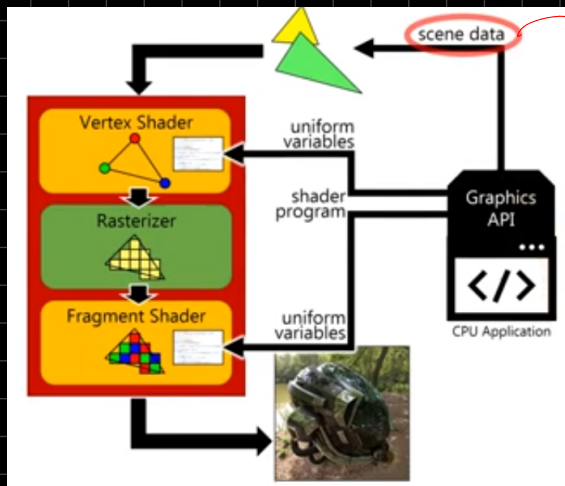


Pipeline Gráfico



Scene Data : (Son los datos Primitivos)

- GL_POINTS
- GL_LINES
- GL_LINE_STRIP
- GL_LINE_LOOP
- GL_TRIANGLES
- GL_TRIANGLE_STRIP
- GL_TRIANGLE_FAN

Web GL Primitivas

- Primitive Type → Pueden ser cualquiera de los datos de escena
- Lista de Vertices : Cae van a definir esa primitiva

- ↳ Vertex Position
- ↳ Vertex Colors
- ↳ Vertex ...
- ↳ Vertex ...
- ↳ Vertex ...

vertex
Attributes

Los vertices son contenedores de información

• Vertex Attributes

positions				colors				
0	x	y	z	0	r	g	b	a
1	x	y	z	1	r	g	b	a
2	x	y	z	2	r	g	b	a
3	x	y	z	3	r	g	b	a
4	x	y	z	4	r	g	b	a
5	x	y	z	5	r	g	b	a
6	x	y	z	6	r	g	b	a

Son los datos que va a usar la GPU puntualmente al vertex Shader

Podemos crear `var positions = [...]` y `var colors = [...]`

```
var positions = [
  -0.8, 0.4, 0,
  0.8, 0.4, 0,
  0.8, -0.4, 0,
  -0.8, 0.4, 0,
  0.8, -0.4, 0,
  -0.8, -0.4, 0
];

var colors = [
  1, 0, 0, 1,
  0, 1, 0, 1,
  0, 0, 1, 1,
  1, 0, 0, 1,
  0, 0, 1, 1,
  1, 0, 1, 1
];
```


Pipeline Gráfico Programable

Def: Secuencia de etapas para crear una representación 2D rasterizada de una escena 3D

Programable vs Fixed

Modelo de cámara Virtual

Escena 3D = Luces + Superficies + Materiales + Cámara

Modelo 3D → Modelos hechos con triángulos

Más Triángulos → Mayor Fidelidad

Modelo 3D de la tetera

Materiales (Pixel Shaders) Definen un color/textura a cada Pixel

Pipeline Gráfico

Secuencia de etapas interconectadas que procesan datos para obtener una imagen.

Data Stream

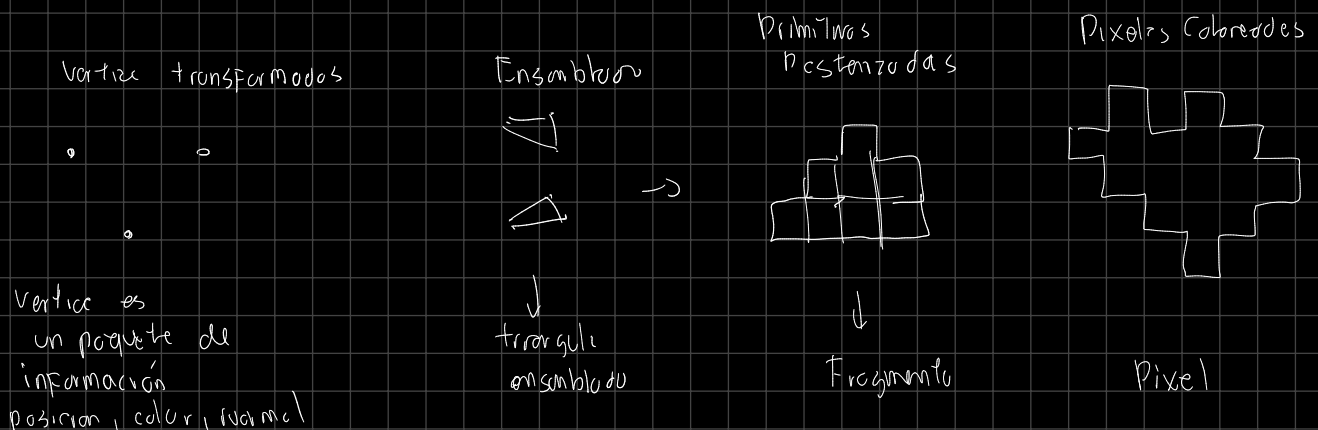
$[x_1, y_1, z_1, x_2, y_2, z_2, \dots, x_n, y_n, z_n]$
 $[r_1, g_1, b_1, r_2, g_2, b_2, \dots, r_n, g_n, b_n]$
 $[u_1, v_1, u_2, v_2, \dots, u_n, v_n]$ →

Rastering Pipeline

Comandos

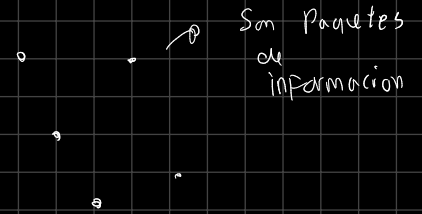
GL_TRIANGLES

La GPU posee cientos de núcleos capaces de ejecutar operaciones en paralelo



Vertex Data

Texture	S T
Posición	X Y Z
Color	R G B A



Rasterizador

Mundo Vectorial

Raster world

Convierte datos vectoriales a pixeles. Esta etapa no es programable.